

Lecture 11: Time & Space Complexity Part 2

PROBLEM STATEMENT

In this lecture, we will deeply analyze **loop-based time complexity**. We will look at different loop patterns — exponential increment, nested loops, variable inner loops — and derive their exact time complexities. We will also compare **Best, Average, Worst Case** and **Big-O, Omega, Theta** notations.

INTUITION

The easiest way to calculate time complexity is to count how many times the code inside the loop executes. If each iteration costs 1 unit, then the total cost is the number of iterations. We then find the highest power of n that fits the total iterations, and that's the time complexity.



Example 1: Exponential Increment Loop (Base 2)

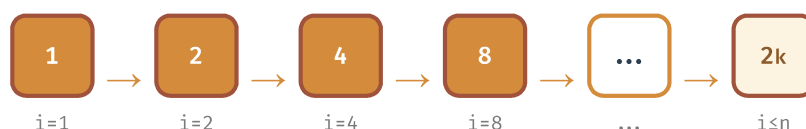
Code Snippet

```
// Loop with i = i * 2
for (int i = 1; i < n; i = i * 2) {
    cout << "hello";
}
```

Step-by-Step Dry Run

DRY RUN / VISUALIZATION

Value of i in each iteration:



Iteration count: $1, 2, 4, 8, \dots, 2^k$ until $2^k \leq n$

Mathematical Derivation

$$2^k \leq n$$

$$\log_2(2^k) \leq \log_2(n)$$

$$k \cdot \log_2(2) \leq \log_2(n)$$

$$k \leq \log_2(n)$$

$$\text{T.C.} = O(\log_2 n)$$

Key Insight: Jab loop variable ko har baar **2 se multiply** karte hain, toh total iterations $\log_2 n$ hoti hain. Kyunki har step pe value double ho rahi hai, isliye bahut jaldi n ke paas pahunch jaate hain.



Example 2: Exponential Increment Loop (Base 3)

Code Snippet

```
// Loop with i = i * 3
for (int i = 1; i < n; i = i * 3) {
    cout << "hello";
}
```

Mathematical Derivation

$$3^k \leq n$$

$$k \leq \log_3(n)$$

$$\text{T.C.} = O(\log_3 n)$$

BEST PRACTICE / OPTIMIZATION

Base change karne se complexity class same rehti hai — $\log_3 n$ aur $\log_2 n$ dono $O(\log n)$ mein aate hain. Lekin exact count ke liye base matter karta hai. Interview mein generally $O(\log n)$ likhna sufficient hai.



Example 3: Halving Loop (While i > 0)

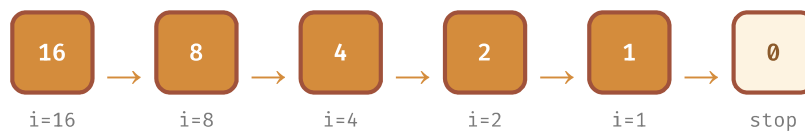
Code Snippet

```
int i = n;
while (i > 0) {
    cout << "hello";
    i = i / 2;
}
```

Step-by-Step Dry Run

DRY RUN / VISUALIZATION

Assume `n = 16` :



Har step pe `n` ka aadha ho raha hai: `n, n/2, n/4, n/8, ...`

Mathematical Derivation

$$n / 2^k \leq 1 \quad (\text{jab tak } i > 0)$$

$$n \leq 2^k$$

$$\log_2(n) \leq k$$

$$\text{T.C.} = O(\log_2 n)$$

IMPORTANT

Exponential increment (`i = i * 2`) aur halving decrement (`i = i / 2`) dono same complexity dete hain — **$O(\log n)$** . Dono cases mein input size exponentially reduce ya increase ho rahi hai.



Example 4: Triple Nested Loop ($O(n^3)$)

Code Snippet

```
// Three nested loops, each running n times
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            cout << "hello";
        }
    }
}
```

Complexity Analysis

COMPLEXITY ANALYSIS

Outer loop n baar chalega. Har outer iteration ke andar middle loop n baar. Har middle iteration ke andar inner loop n baar.

Total iterations = $n \times n \times n$

T.C. = $O(n^3)$



Example 5: Variable Inner Loop (Dependent on Outer)

Code Snippet

```
// Inner loop depends on outer loop variable i
for (int i = 1; i < n; i++) {
    for (int j = 0; j < i * i; j++) {
        cout << "hello";
    }
}
```

Step-by-Step Dry Run

DRY RUN / VISUALIZATION

Outer i	Inner j runs	Iterations
i = 1	j = 0 to 0	1 time
i = 2	j = 0 to 3	4 times
i = 3	j = 0 to 8	9 times
i = 4	j = 0 to 15	16 times
...	...	...
i = n-1	j = 0 to $(n-1)^2 - 1$	$(n-1)^2$ times

Mathematical Derivation

Sum = $1^2 + 2^2 + 3^2 + 4^2 + \dots + (n-1)^2$

Formula: $\sum k^2 = n(n+1)(2n+1) / 6$

For (n-1) terms: $(n-1)(n)(2n-1) / 6$

Expand: $(2n^3 - 3n^2 + n) / 6$

Dominant term: $2n^3 / 6 = n^3 / 3$

T.C. = $O(n^3)$

Key Insight: "Sabse badi value ko pakad ke chalna" — jab bhi polynomial sum aaye, toh sabse highest degree wale term ko utha lo. Constants aur lower degree terms ko ignore kar do. Yahi Big-O analysis ka core principle hai.



Best, Average & Worst Case Analysis

Understanding Through Linear Search

```
int arr[] = {10, 20, 17, 9, 8, 11};
int element = 10;

for (int i = 0; i < n; i++) {
    if (arr[i] == element) {
        cout << "Yes";
        return 0;
    }
}
cout << "No";
```

BEST CASE

Element pehle index pe mil jaye

$O(1)$

AVERAGE CASE

Element beech mein kahi mile

$O(n)$

WORST CASE

Element last mein mile ya na mile

$O(n)$

INTERVIEW TIP

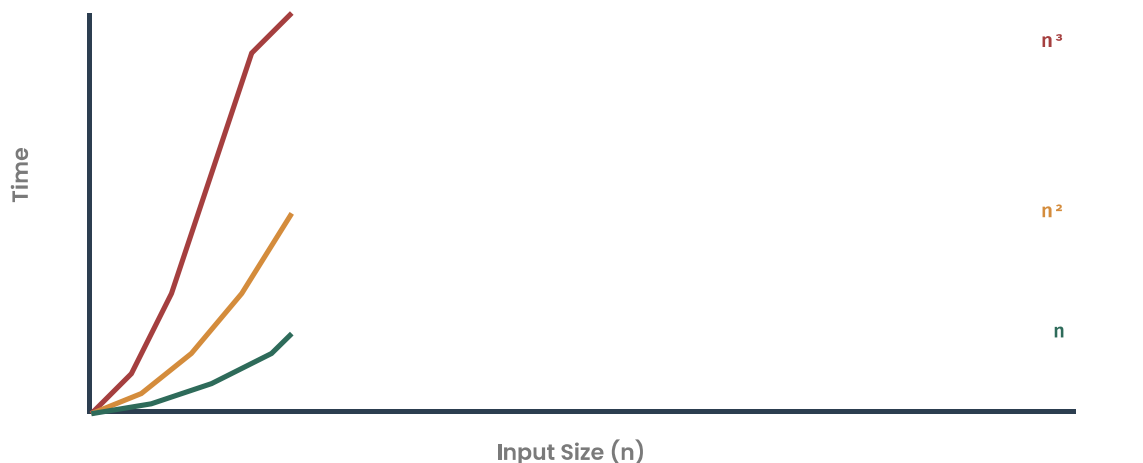
Interview mein hamesha Worst Case complexity hi batate hain. Kyunki interviewer yeh dekhna chahta hai ki algorithm sabse bure scenario mein kitna time lega. Best case toh rarely hota hai, aur average case ka assumption input distribution pe depend karta hai. Isliye safe answer hamesha $O(n)$ (worst case) batana.



Asymptotic Notations: Big-O, Omega & Theta

Visual Comparison

Growth Rate Comparison



Notation Deep Dive

O Big-O (Upper Bound)

Big-O notation **sabse zyada time** batata hai — matlab algorithm kabhi bhi isse zyada time nahi lega. Interview mein hamesha Big-O pucha jata hai kyunki yeh **upper bound** provide karta hai.

Example: Agar T.C. = n^3 hai, toh Big-O = $O(n^3)$. Algorithm n^3 se zyada time nahi lega.

Ω Omega (Lower Bound)

Omega notation **sabse kam time** batata hai — matlab algorithm itna time toh minimum lega hi. Yeh **lower bound** hota hai. Agar T.C. = n^3 hai, toh $\Omega(n^3)$ bhi hoga kyunki kam se kam itna time toh lagega.

Example: $\Omega(n^3)$ ka matlab algorithm n^3 se kam time mein complete nahi ho sakta.

Θ Theta (Tight Bound)

Theta tab use hota hai jab **exact time complexity** pata ho. Jab upper bound aur lower bound same hote hain, tab Theta use karte hain. Lekin practically, **Theta sirf tab use karte hain jab T.C. exactly known ho**.

Real World: Mostly Big-O hi use hota hai kyunki exact bound nikalna mushkil hota hai.

IMPORTANT

Teen notations — Big-O, Omega, Theta — mainly **confusion ke liye** rakhe gaye hain. Interview aur competitive programming mein **sirf Big-O** hi use karte hain. Kyunki Big-O mein hum hamesha **upper bound** consider karte hain, jo sabse safe estimate hota hai.

Notation Summary Table

Notation	Name	Bound Type	Meaning	Usage
O	Big-O	Upper Bound	Sabse zyada time (equal ya usse zyada)	Interviews, CP
Ω	Omega	Lower Bound	Sabse kam time (equal ya usse kam nahi)	Theoretical analysis
Θ	Theta	Tight Bound	Exact time complexity	Jab exact pata ho

Key Insight: $O(n) < O(n^2) < O(n^3)$ — yeh growth order hai. Graph mein dekh sakte hain ki n^3 sabse tezi se upar ja raha hai. Isliye jab bhi complexity compare karo, toh sabse bade exponent wale term ko pakdo.



Complexity Comparison Cheat Sheet

Loop Pattern	Code Structure	Time Complexity
Single linear loop	<code>for(i=0; i<n; i++)</code>	$O(n)$
Double nested loop	<code>for(i) { for(j) }</code>	$O(n^2)$
Triple nested loop	<code>for(i) { for(j) { for(k) } }</code>	$O(n^3)$
Exponential increment ($\times 2$)	<code>i = i * 2</code>	$O(\log_2 n)$
Exponential increment ($\times 3$)	<code>i = i * 3</code>	$O(\log_3 n)$
Halving decrement	<code>i = i / 2</code>	$O(\log_2 n)$
Variable inner loop	<code>j < i*i</code>	$O(n^2)$



Interview Tips & Common Pitfalls

INTERVIEW TIP

- **Hamesha worst case complexity** batana — interviewer ko yeh pasand hai.
- Nested loops ke case mein **multiply** karo iterations ko, add nahi.
- Exponential loops ($\times 2$, $\times 3$, $/2$) ka base matter nahi karta class ke liye — sab $O(\log n)$ mein aate hain.
- Sum of squares formula yaad rakho: $\sum k^2 = n(n+1)(2n+1)/6$
- Big-O, Omega, Theta mein sirf **Big-O** hi practically use hota hai.

EDGE CASE

- Agar $n = 0$ ho toh loop kabhi execute nahi hoga — complexity technically $O(1)$ (constant time check).
- Integer overflow ka dhyan rakho: $i = i * 2$ mein agar i int hai toh overflow ho sakta hai bahut bade n ke liye.
- `while(i > 0)` mein agar i negative ho jaye toh infinite loop — isliye unsigned ya proper check zaroori hai.



Key Takeaways

TAKEAWAY

1. **Loop analysis** ka basic rule: har iteration ka cost 1 maan ke total iterations count karo.
2. Exponential increment/decrement loops ($\times 2$, $\times 3$, $/2$) dete hain $O(\log n)$ complexity.
3. Nested independent loops multiply hote hain: $n \times n \times n = O(n^3)$.
4. Dependent inner loops mein **series sum** nikaal ke highest degree term pakdo.
5. **Sabse badi value ko pakad ke chalna** — lower order terms aur constants ko ignore karo.
6. Interview mein hamesha **Worst Case + Big-O** batana.
7. Big-O = Upper Bound, Omega = Lower Bound, Theta = Tight Bound. Practically sirf Big-O use hota hai.